

“The Game Beneath the Game: The Evolution of Cheats and Anti-Cheat Systems in Counter-Strike 2”

Author: Pablo Valiente

Supervisor: Dr. Antonio Nappa

Universidad Carlos III de Madrid

Academic Year: 2024–2025

Table of Contents

1. Motivation and Context
2. Cheating: History, Ecosystem, and Ethics
3. Anti-Cheat Technologies and Milestones
4. Case Study: Counter-Strike 2
5. Technical Analysis: Injection and Evasion
6. Game Logic Manipulation and Visual Features
7. Final Reflections and Future Directions

Table of Contents

1. Motivation and Context
2. Cheating: History, Ecosystem, and Ethics
3. Anti-Cheat Technologies and Milestones
4. Case Study: Counter-Strike 2
5. Technical Analysis: Injection and Evasion
6. Game Logic Manipulation and Visual Features
7. Final Reflections and Future Directions

Table of Contents

1. Motivation and Context
2. Cheating: History, Ecosystem, and Ethics
3. Anti-Cheat Technologies and Milestones
4. Case Study: Counter-Strike 2
5. Technical Analysis: Injection and Evasion
6. Game Logic Manipulation and Visual Features
7. Final Reflections and Future Directions

Table of Contents

1. Motivation and Context
2. Cheating: History, Ecosystem, and Ethics
3. Anti-Cheat Technologies and Milestones
4. Case Study: Counter-Strike 2
5. Technical Analysis: Injection and Evasion
6. Game Logic Manipulation and Visual Features
7. Final Reflections and Future Directions

Table of Contents

1. Motivation and Context
2. Cheating: History, Ecosystem, and Ethics
3. Anti-Cheat Technologies and Milestones
4. Case Study: Counter-Strike 2
5. Technical Analysis: Injection and Evasion
6. Game Logic Manipulation and Visual Features
7. Final Reflections and Future Directions

Table of Contents

1. Motivation and Context
2. Cheating: History, Ecosystem, and Ethics
3. Anti-Cheat Technologies and Milestones
4. Case Study: Counter-Strike 2
5. Technical Analysis: Injection and Evasion
6. Game Logic Manipulation and Visual Features
7. Final Reflections and Future Directions

Table of Contents

1. Motivation and Context
2. Cheating: History, Ecosystem, and Ethics
3. Anti-Cheat Technologies and Milestones
4. Case Study: Counter-Strike 2
5. Technical Analysis: Injection and Evasion
6. Game Logic Manipulation and Visual Features
7. Final Reflections and Future Directions

Motivation

My interest in this topic began around six years ago, driven by a passion for competitive video games.

- **Curiosity:** Encountering cheaters in ranked matches often led me to wonder: *"How did they do it?"*
- **Exploration:** This question gradually introduced me to reverse engineering, memory analysis, and anti-cheat evasion.
- **Background:** Over the years, I acquired technical knowledge through communities, experiments, and academic training.
- **Objective:** To merge this journey with a rigorous academic study by analyzing and implementing real cheats to test detection systems.

Motivation

My interest in this topic began around six years ago, driven by a passion for competitive video games.

- **Curiosity:** Encountering cheaters in ranked matches often led me to wonder: *"How did they do it?"*
- **Exploration:** This question gradually introduced me to reverse engineering, memory analysis, and anti-cheat evasion.
- **Background:** Over the years, I acquired technical knowledge through communities, experiments, and academic training.
- **Objective:** To merge this journey with a rigorous academic study by analyzing and implementing real cheats to test detection systems.

Motivation

My interest in this topic began around six years ago, driven by a passion for competitive video games.

- **Curiosity:** Encountering cheaters in ranked matches often led me to wonder: *"How did they do it?"*
- **Exploration:** This question gradually introduced me to reverse engineering, memory analysis, and anti-cheat evasion.
- **Background:** Over the years, I acquired technical knowledge through communities, experiments, and academic training.
- **Objective:** To merge this journey with a rigorous academic study by analyzing and implementing real cheats to test detection systems.

Motivation

My interest in this topic began around six years ago, driven by a passion for competitive video games.

- **Curiosity:** Encountering cheaters in ranked matches often led me to wonder: *"How did they do it?"*
- **Exploration:** This question gradually introduced me to reverse engineering, memory analysis, and anti-cheat evasion.
- **Background:** Over the years, I acquired technical knowledge through communities, experiments, and academic training.
- **Objective:** To merge this journey with a rigorous academic study by analyzing and implementing real cheats to test detection systems.

Motivation and Context

The Evolution of Cheating: History, Ecosystem, and Ethics

The Rise of Anti-Cheat: Technologies and Milestones

Counter-Strike 2: A Case Study in Cheating Dynamics

Code Injection and Cheat Delivery Mechanisms

Cheat Implementation: Hooking, Logic, and Features

Reflections, Implications, and Future Work

Understanding Cheating in Online Games

Cheating undermines fairness and security in competitive online games.

- **Definition:** Any software or hardware manipulation that grants unfair advantage.
- **Anti-Cheat:** Monitors and enforces integrity at runtime, acting like an **EDR** by detecting anomalous behavior, unauthorized memory access, or code injection in protected processes.
- **Spectrum:** From basic aimbots to advanced GUI Drawings.
- **Impact:** Threatens matchmaking integrity and industry revenue.
- **Context:** Now part of the broader cybersecurity landscape.

Understanding Cheating in Online Games

Cheating undermines fairness and security in competitive online games.

- **Definition:** Any software or hardware manipulation that grants unfair advantage.
- **Anti-Cheat:** Monitors and enforces integrity at runtime, acting like an **EDR** by detecting anomalous behavior, unauthorized memory access, or code injection in protected processes.
- **Spectrum:** From basic aimbots to advanced GUI Drawings.
- **Impact:** Threatens matchmaking integrity and industry revenue.
- **Context:** Now part of the broader cybersecurity landscape.

Understanding Cheating in Online Games

Cheating undermines fairness and security in competitive online games.

- **Definition:** Any software or hardware manipulation that grants unfair advantage.
- **Anti-Cheat:** Monitors and enforces integrity at runtime, acting like an **EDR** by detecting anomalous behavior, unauthorized memory access, or code injection in protected processes.
- **Spectrum:** From basic aimbots to advanced GUI Drawings.
- **Impact:** Threatens matchmaking integrity and industry revenue.
- **Context:** Now part of the broader cybersecurity landscape.

Understanding Cheating in Online Games

Cheating undermines fairness and security in competitive online games.

- **Definition:** Any software or hardware manipulation that grants unfair advantage.
- **Anti-Cheat:** Monitors and enforces integrity at runtime, acting like an **EDR** by detecting anomalous behavior, unauthorized memory access, or code injection in protected processes.
- **Spectrum:** From basic aimbots to advanced GUI Drawings.
- **Impact:** Threatens matchmaking integrity and industry revenue.
- **Context:** Now part of the broader cybersecurity landscape.

Understanding Cheating in Online Games

Cheating undermines fairness and security in competitive online games.

- **Definition:** Any software or hardware manipulation that grants unfair advantage.
- **Anti-Cheat:** Monitors and enforces integrity at runtime, acting like an **EDR** by detecting anomalous behavior, unauthorized memory access, or code injection in protected processes.
- **Spectrum:** From basic aimbots to advanced GUI Drawings.
- **Impact:** Threatens matchmaking integrity and industry revenue.
- **Context:** Now part of the broader cybersecurity landscape.

Origins of Cheating: From Debugging to Subversion

Cheating emerged from tools meant for testing and evolved into manipulation.

- **Debug Codes:** Became public cheat codes (e.g., Konami Code).
- **GameShark:** Modified RAM directly on console hardware.
- **Cheat Engine:** Enabled runtime memory editing on PC.
- **x64dbg and ReClass.NET:** Allowed advanced memory mapping, pointer tracing, and class reconstruction.
- **Advanced Tooling:** Debuggers like x64dbg, memory editors, and tools such as ReClass.NET enabled deep inspection of structures and dynamic memory regions.

Origins of Cheating: From Debugging to Subversion

Cheating emerged from tools meant for testing and evolved into manipulation.

- **Debug Codes:** Became public cheat codes (e.g., Konami Code).
- **GameShark:** Modified RAM directly on console hardware.
- **Cheat Engine:** Enabled runtime memory editing on PC.
- **x64dbg and ReClass.NET:** Allowed advanced memory mapping, pointer tracing, and class reconstruction.
- **Advanced Tooling:** Debuggers like x64dbg, memory editors, and tools such as ReClass.NET enabled deep inspection of structures and dynamic memory regions.

Origins of Cheating: From Debugging to Subversion

Cheating emerged from tools meant for testing and evolved into manipulation.

- **Debug Codes:** Became public cheat codes (e.g., Konami Code).
- **GameShark:** Modified RAM directly on console hardware.
- **Cheat Engine:** Enabled runtime memory editing on PC.
- **x64dbg and ReClass.NET:** Allowed advanced memory mapping, pointer tracing, and class reconstruction.
- **Advanced Tooling:** Debuggers like x64dbg, memory editors, and tools such as ReClass.NET enabled deep inspection of structures and dynamic memory regions.

Origins of Cheating: From Debugging to Subversion

Cheating emerged from tools meant for testing and evolved into manipulation.

- **Debug Codes:** Became public cheat codes (e.g., Konami Code).
- **GameShark:** Modified RAM directly on console hardware.
- **Cheat Engine:** Enabled runtime memory editing on PC.
- **x64dbg and ReClass.NET:** Allowed advanced memory mapping, pointer tracing, and class reconstruction.
- **Advanced Tooling:** Debuggers like x64dbg, memory editors, and tools such as ReClass.NET enabled deep inspection of structures and dynamic memory regions.

Origins of Cheating: From Debugging to Subversion

Cheating emerged from tools meant for testing and evolved into manipulation.

- **Debug Codes:** Became public cheat codes (e.g., Konami Code).
- **GameShark:** Modified RAM directly on console hardware.
- **Cheat Engine:** Enabled runtime memory editing on PC.
- **x64dbg and ReClass.NET:** Allowed advanced memory mapping, pointer tracing, and class reconstruction.
- **Advanced Tooling:** Debuggers like x64dbg, memory editors, and tools such as ReClass.NET enabled deep inspection of structures and dynamic memory regions.

From Hacks to Services: The Turning Point

The early 2010s marked a major shift in how cheats were created, distributed, and perceived.

- **Community Forums:** Platforms like UnKnoWnCheaTs fostered knowledge sharing and tutorials.
- **Toolchains:** Public tools and open-source loaders lowered the barrier to entry.
- **Subscription Models:** Private cheats became commercialized with regular updates and support.
- **User Demand:** Players sought not just to win, but to dominate with minimal effort, fueling a sustainable market.

From Hacks to Services: The Turning Point

The early 2010s marked a major shift in how cheats were created, distributed, and perceived.

- **Community Forums:** Platforms like UnKnoWnCheaTs fostered knowledge sharing and tutorials.
- **Toolchains:** Public tools and open-source loaders lowered the barrier to entry.
- **Subscription Models:** Private cheats became commercialized with regular updates and support.
- **User Demand:** Players sought not just to win, but to dominate with minimal effort, fueling a sustainable market.

From Hacks to Services: The Turning Point

The early 2010s marked a major shift in how cheats were created, distributed, and perceived.

- **Community Forums:** Platforms like UnKnoWnCheaTs fostered knowledge sharing and tutorials.
- **Toolchains:** Public tools and open-source loaders lowered the barrier to entry.
- **Subscription Models:** Private cheats became commercialized with regular updates and support.
- **User Demand:** Players sought not just to win, but to dominate with minimal effort, fueling a sustainable market.

From Hacks to Services: The Turning Point

The early 2010s marked a major shift in how cheats were created, distributed, and perceived.

- **Community Forums:** Platforms like UnKnoWnCheaTs fostered knowledge sharing and tutorials.
- **Toolchains:** Public tools and open-source loaders lowered the barrier to entry.
- **Subscription Models:** Private cheats became commercialized with regular updates and support.
- **User Demand:** Players sought not just to win, but to dominate with minimal effort, fueling a sustainable market.

Cheating Today: A Professionalized Ecosystem

Cheating has evolved from hobbyist tools to a mature ecosystem that mirrors legitimate software development.

- **Commercialization:** Cheat providers operate like startups, offering licensing, tiered features, and customer support.
- **Stealth Technology:** Use of polymorphism, virtualization, and driver-level code to evade detection.
- **Infrastructure:** Loaders, obfuscators, HWID spoofers, and encrypted update channels ensure persistence.
- **Community:** Development is sustained by highly technical reverse engineers who are constantly reversing every update in the game.

Cheating Today: A Professionalized Ecosystem

Cheating has evolved from hobbyist tools to a mature ecosystem that mirrors legitimate software development.

- **Commercialization:** Cheat providers operate like startups, offering licensing, tiered features, and customer support.
- **Stealth Technology:** Use of polymorphism, virtualization, and driver-level code to evade detection.
- **Infrastructure:** Loaders, obfuscators, HWID spoofers, and encrypted update channels ensure persistence.
- **Community:** Development is sustained by highly technical reverse engineers who are constantly reversing every update in the game.

Cheating Today: A Professionalized Ecosystem

Cheating has evolved from hobbyist tools to a mature ecosystem that mirrors legitimate software development.

- **Commercialization:** Cheat providers operate like startups, offering licensing, tiered features, and customer support.
- **Stealth Technology:** Use of polymorphism, virtualization, and driver-level code to evade detection.
- **Infrastructure:** Loaders, obfuscators, HWID spoofers, and encrypted update channels ensure persistence.
- **Community:** Development is sustained by highly technical reverse engineers who are constantly reversing every update in the game.

Cheating Today: A Professionalized Ecosystem

Cheating has evolved from hobbyist tools to a mature ecosystem that mirrors legitimate software development.

- **Commercialization:** Cheat providers operate like startups, offering licensing, tiered features, and customer support.
- **Stealth Technology:** Use of polymorphism, virtualization, and driver-level code to evade detection.
- **Infrastructure:** Loaders, obfuscators, HWID spoofers, and encrypted update channels ensure persistence.
- **Community:** Development is sustained by highly technical reverse engineers who are constantly reversing every update in the game.

Outline

Motivation and Context

The Evolution of Cheating: History, Ecosystem, and Ethics

The Rise of Anti-Cheat: Technologies and Milestones

Counter-Strike 2: A Case Study in Cheating Dynamics

Code Injection and Cheat Delivery Mechanisms

Cheat Implementation: Hooking, Logic, and Features

Reflections, Implications, and Future Work

Anti-Cheat Systems Timeline

Anti-cheat systems have evolved from basic client-side tools to complex, kernel-level detectors. Each marked a significant shift in visibility, access, or enforcement.

System	Year	Mode	Milestone or Innovation
PunkBuster	2000	User	First real-time memory scan and screenshot system
VAC	2002	User	Delayed ban model to obfuscate detection methods
BattleEye	2004	Kernel	Early adoption of kernel-mode drivers
FairFight	2012	Server	Behavioral/statistical detection at server level
EAC	2018	Kernel	Real-time scanning with driver-level protection
Vanguard	2020	Kernel	Persistent always-on kernel driver (controversial)
VAC Live	2023	Hybrid	Real-time detection with low-latency enforcement

Why FPS Games Are Especially Vulnerable

First-person shooters present unique architectural weaknesses that make them highly susceptible to game manipulation.

- **Client-Side Rendering:** The game client must access enemy positions to render them, even through walls.
- **Memory Residency:** Positional and player state data exist in memory and can be externally read or modified.
- **Contrast With Strategy Games:** RTS or turn-based titles store game logic server-side, making manipulation far more difficult.
- **Exploitation Vector:** These FPS traits enable features like ESP, aimbots, and recoil control by hijacking user input or memory data.

Why FPS Games Are Especially Vulnerable

First-person shooters present unique architectural weaknesses that make them highly susceptible to game manipulation.

- **Client-Side Rendering:** The game client must access enemy positions to render them, even through walls.
- **Memory Residency:** Positional and player state data exist in memory and can be externally read or modified.
- **Contrast With Strategy Games:** RTS or turn-based titles store game logic server-side, making manipulation far more difficult.
- **Exploitation Vector:** These FPS traits enable features like ESP, aimbots, and recoil control by hijacking user input or memory data.

Why FPS Games Are Especially Vulnerable

First-person shooters present unique architectural weaknesses that make them highly susceptible to game manipulation.

- **Client-Side Rendering:** The game client must access enemy positions to render them, even through walls.
- **Memory Residency:** Positional and player state data exist in memory and can be externally read or modified.
- **Contrast With Strategy Games:** RTS or turn-based titles store game logic server-side, making manipulation far more difficult.
- **Exploitation Vector:** These FPS traits enable features like ESP, aimbots, and recoil control by hijacking user input or memory data.

Why FPS Games Are Especially Vulnerable

First-person shooters present unique architectural weaknesses that make them highly susceptible to game manipulation.

- **Client-Side Rendering:** The game client must access enemy positions to render them, even through walls.
- **Memory Residency:** Positional and player state data exist in memory and can be externally read or modified.
- **Contrast With Strategy Games:** RTS or turn-based titles store game logic server-side, making manipulation far more difficult.
- **Exploitation Vector:** These FPS traits enable features like ESP, aimbots, and recoil control by hijacking user input or memory data.

Why Counter-Strike 2?

Counter-Strike 2 represents a prime environment to study modern cheating.

- **Large Player Base:** Over 1 million monthly active users in 2024–2025.
- **High Competitive Stakes:** Ranked ladders, esports scene, and skin economy intensify incentives.
- **Persistent Cheating Problem:** VAC bans, report abuse, and public complaints highlight widespread issues.
- **Technical Maturity:** CS2 is built on Source 2 with new anti-cheat features and detection paradigms.
- **Public Data:** Rich dataset from platforms like SteamCharts and CSStats enables evidence-based analysis.

Why Counter-Strike 2?

Counter-Strike 2 represents a prime environment to study modern cheating.

- **Large Player Base:** Over 1 million monthly active users in 2024–2025.
- **High Competitive Stakes:** Ranked ladders, esports scene, and skin economy intensify incentives.
- **Persistent Cheating Problem:** VAC bans, report abuse, and public complaints highlight widespread issues.
- **Technical Maturity:** CS2 is built on Source 2 with new anti-cheat features and detection paradigms.
- **Public Data:** Rich dataset from platforms like SteamCharts and CSStats enables evidence-based analysis.

Why Counter-Strike 2?

Counter-Strike 2 represents a prime environment to study modern cheating.

- **Large Player Base:** Over 1 million monthly active users in 2024–2025.
- **High Competitive Stakes:** Ranked ladders, esports scene, and skin economy intensify incentives.
- **Persistent Cheating Problem:** VAC bans, report abuse, and public complaints highlight widespread issues.
- **Technical Maturity:** CS2 is built on Source 2 with new anti-cheat features and detection paradigms.
- **Public Data:** Rich dataset from platforms like SteamCharts and CSStats enables evidence-based analysis.

Why Counter-Strike 2?

Counter-Strike 2 represents a prime environment to study modern cheating.

- **Large Player Base:** Over 1 million monthly active users in 2024–2025.
- **High Competitive Stakes:** Ranked ladders, esports scene, and skin economy intensify incentives.
- **Persistent Cheating Problem:** VAC bans, report abuse, and public complaints highlight widespread issues.
- **Technical Maturity:** CS2 is built on Source 2 with new anti-cheat features and detection paradigms.
- **Public Data:** Rich dataset from platforms like SteamCharts and CSStats enables evidence-based analysis.

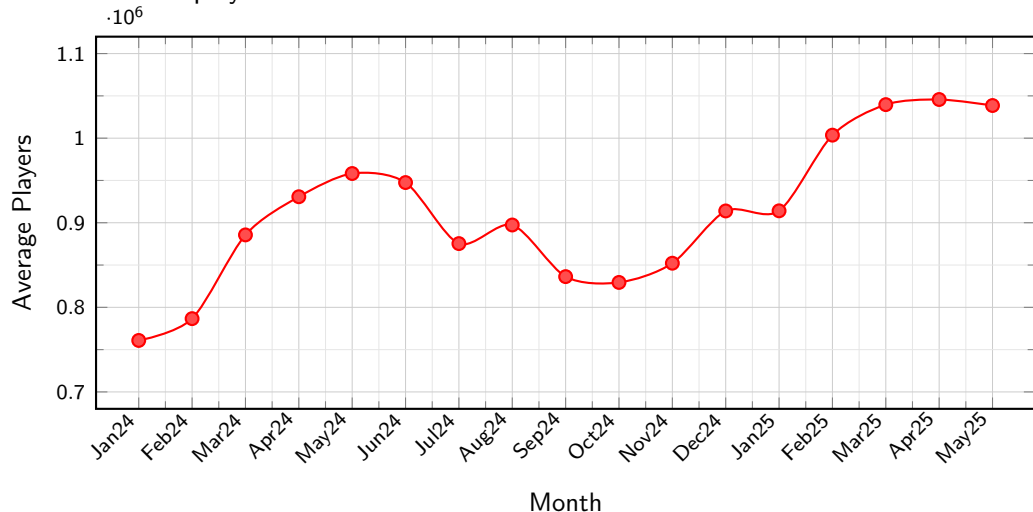
Why Counter-Strike 2?

Counter-Strike 2 represents a prime environment to study modern cheating.

- **Large Player Base:** Over 1 million monthly active users in 2024–2025.
- **High Competitive Stakes:** Ranked ladders, esports scene, and skin economy intensify incentives.
- **Persistent Cheating Problem:** VAC bans, report abuse, and public complaints highlight widespread issues.
- **Technical Maturity:** CS2 is built on Source 2 with new anti-cheat features and detection paradigms.
- **Public Data:** Rich dataset from platforms like SteamCharts and CSStats enables evidence-based analysis.

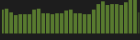
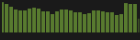
CS2 Player Base Growth (2024–2025)

To contextualize the scale of CS2's ecosystem, the following graph illustrates the evolution of its player base based on SteamCharts data.



Steam Top 10: July 7, 2025

Top Games By Current Players

	Name	Current Players	Last 30 Days	Peak Players	Hours Played
1.	Counter-Strike 2	1,222,782		1,725,815	729,749,131
2.	PUBG: BATTLEGROUNDS	581,616		778,327	226,931,487
3.	Dota 2	508,454		648,095	300,471,647
4.	Bongo Cat	169,028		192,501	110,247,318
5.	Delta Force	164,154		177,775	60,062,183
6.	Apex Legends	150,271		250,796	73,537,334
7.	Stardew Valley	133,590		156,589	47,739,319
8.	Rust	129,532		187,345	73,879,862
9.	Wallpaper Engine	120,513		138,702	61,757,771
10.	Grand Theft Auto V Legacy	90,856		112,862	53,501,676

[More](#)

Motivation and Context

The Evolution of Cheating: History, Ecosystem, and Ethics

The Rise of Anti-Cheat: Technologies and Milestones

Counter-Strike 2: A Case Study in Cheating Dynamics

Code Injection and Cheat Delivery Mechanisms

Cheat Implementation: Hooking, Logic, and Features

Reflections, Implications, and Future Work

VAC: Valve's Anti-Cheat Strategy

Since the early days of Counter-Strike, Valve has maintained its own anti-cheat system, VAC (Valve Anti-Cheat).

- **Consistency:** VAC has been used since CS 1.6 and evolved alongside the series.
- **User Mode Limitation:** VAC operates entirely in user space, unlike newer kernel-mode anticheats.
- **Detection Philosophy:** Prefers silent detection with delayed banwaves to avoid tipping off cheat developers.
- **Upcoming Evolution:** Valve announced integration of **machine learning models** into VAC Live to detect behavioral patterns and cheating heuristics in real time.

VAC: Valve's Anti-Cheat Strategy

Since the early days of Counter-Strike, Valve has maintained its own anti-cheat system, VAC (Valve Anti-Cheat).

- **Consistency:** VAC has been used since CS 1.6 and evolved alongside the series.
- **User Mode Limitation:** VAC operates entirely in user space, unlike newer kernel-mode anticheats.
- **Detection Philosophy:** Prefers silent detection with delayed banwaves to avoid tipping off cheat developers.
- **Upcoming Evolution:** Valve announced integration of **machine learning models** into VAC Live to detect behavioral patterns and cheating heuristics in real time.

VAC: Valve's Anti-Cheat Strategy

Since the early days of Counter-Strike, Valve has maintained its own anti-cheat system, VAC (Valve Anti-Cheat).

- **Consistency:** VAC has been used since CS 1.6 and evolved alongside the series.
- **User Mode Limitation:** VAC operates entirely in user space, unlike newer kernel-mode anticheats.
- **Detection Philosophy:** Prefers silent detection with delayed banwaves to avoid tipping off cheat developers.
- **Upcoming Evolution:** Valve announced integration of **machine learning models** into VAC Live to detect behavioral patterns and cheating heuristics in real time.

VAC: Valve's Anti-Cheat Strategy

Since the early days of Counter-Strike, Valve has maintained its own anti-cheat system, VAC (Valve Anti-Cheat).

- **Consistency:** VAC has been used since CS 1.6 and evolved alongside the series.
- **User Mode Limitation:** VAC operates entirely in user space, unlike newer kernel-mode anticheats.
- **Detection Philosophy:** Prefers silent detection with delayed banwaves to avoid tipping off cheat developers.
- **Upcoming Evolution:** Valve announced integration of **machine learning models** into VAC Live to detect behavioral patterns and cheating heuristics in real time.

Trust Factor and Detection Challenges

Valve's Trust Factor system, introduced in late 2017, fundamentally changed matchmaking by leveraging a hidden reputation metric to segregate players.

- **Function:** Uses behavioral data, report history, account characteristics, and system metrics to infer player reliability.
- **Segregation Strategy:** Groups suspect cheaters with other suspect cheaters, and legitimate players with each other, to reduce disruptive overlap.
- **Opacity:** The exact calculation is hidden, preventing transparency or dispute of unfair evaluations.

Trust Factor and Detection Challenges

Valve's Trust Factor system, introduced in late 2017, fundamentally changed matchmaking by leveraging a hidden reputation metric to segregate players.

- **Function:** Uses behavioral data, report history, account characteristics, and system metrics to infer player reliability.
- **Segregation Strategy:** Groups suspect cheaters with other suspect cheaters, and legitimate players with each other, to reduce disruptive overlap.
- **Opacity:** The exact calculation is hidden, preventing transparency or dispute of unfair evaluations.

Trust Factor and Detection Challenges

Valve's Trust Factor system, introduced in late 2017, fundamentally changed matchmaking by leveraging a hidden reputation metric to segregate players.

- **Function:** Uses behavioral data, report history, account characteristics, and system metrics to infer player reliability.
- **Segregation Strategy:** Groups suspect cheaters with other suspect cheaters, and legitimate players with each other, to reduce disruptive overlap.
- **Opacity:** The exact calculation is hidden, preventing transparency or dispute of unfair evaluations.

What Actually Affects Trust Factor?

Although Valve keeps the exact algorithm secret, several factors are known or strongly suspected to influence Trust Factor:

- **Account Age and Game Hours:** Older accounts and higher total playtime positively impact Trust Factor.
- **Money Spent:** Purchasing games or items (e.g., skins, Prime status) improves account reputation.
- **Report History:** Frequent reports, especially confirmed ones, lower your Trust Factor.
- **Friend Network:** Being friends with banned or suspicious players may reduce your Trust Factor.

What Actually Affects Trust Factor?

Although Valve keeps the exact algorithm secret, several factors are known or strongly suspected to influence Trust Factor:

- **Account Age and Game Hours:** Older accounts and higher total playtime positively impact Trust Factor.
- **Money Spent:** Purchasing games or items (e.g., skins, Prime status) improves account reputation.
- **Report History:** Frequent reports, especially confirmed ones, lower your Trust Factor.
- **Friend Network:** Being friends with banned or suspicious players may reduce your Trust Factor.

What Actually Affects Trust Factor?

Although Valve keeps the exact algorithm secret, several factors are known or strongly suspected to influence Trust Factor:

- **Account Age and Game Hours:** Older accounts and higher total playtime positively impact Trust Factor.
- **Money Spent:** Purchasing games or items (e.g., skins, Prime status) improves account reputation.
- **Report History:** Frequent reports, especially confirmed ones, lower your Trust Factor.
- **Friend Network:** Being friends with banned or suspicious players may reduce your Trust Factor.

What Actually Affects Trust Factor?

Although Valve keeps the exact algorithm secret, several factors are known or strongly suspected to influence Trust Factor:

- **Account Age and Game Hours:** Older accounts and higher total playtime positively impact Trust Factor.
- **Money Spent:** Purchasing games or items (e.g., skins, Prime status) improves account reputation.
- **Report History:** Frequent reports, especially confirmed ones, lower your Trust Factor.
- **Friend Network:** Being friends with banned or suspicious players may reduce your Trust Factor.

Economic and Psychological Impact

Cheating in CS2 generates profound economic losses and undermines player well-being.

- **Skin Markets: VAC bans** can instantly invalidate cosmetic items worth **thousands of euros**. Locking down those accounts and disabling trade offers.
- **Morale: Rampant cheating** erodes competitive spirit, trust, and perceived fairness.
- **Retention: Frustrated players** abandon the game, shrinking and destabilizing the community.

Economic and Psychological Impact

Cheating in CS2 generates profound economic losses and undermines player well-being.

- **Skin Markets: VAC bans** can instantly invalidate cosmetic items worth **thousands of euros**. Locking down those accounts and disabling trade offers.
- **Morale: Rampant cheating** erodes competitive spirit, trust, and perceived fairness.
- **Retention: Frustrated players** abandon the game, shrinking and destabilizing the community.

Economic and Psychological Impact

Cheating in CS2 generates profound economic losses and undermines player well-being.

- **Skin Markets: VAC bans** can instantly invalidate cosmetic items worth **thousands of euros**. Locking down those accounts and disabling trade offers.
- **Morale: Rampant cheating** erodes competitive spirit, trust, and perceived fairness.
- **Retention: Frustrated players** abandon the game, shrinking and destabilizing the community.

Survey: Cheating as a Critical Threat

A 2018 global survey by Irdeto revealed the widespread negative impact of cheating in Counter-Strike: Global Offensive:

Survey Statement	Percentage
Experienced negative impact due to cheating	60%
Would stop playing a game because of cheaters	77%
Support permanent bans for cheaters	69%
Believe developers are not doing enough	55%

- Highlights long-term damage to player trust and retention.
- Economic implications due to reduced engagement and spending.

Quantifying the Impact: Cheating and Player Statistics

Beyond anecdotal evidence, empirical data helps us understand the scope of cheating in CS2.

- **Player Growth:** Monthly average players increased from 760k to over 1 million (SteamCharts, Jan 2024 – May 2025).
- **Delayed Enforcement:** 28% of VAC bans occur within 24h of last match; average delay is 20 days.
- **Cheating Embedded:** 97% of affected matches had only one banned player, often undetected in real time.
- **No Case Farming Evidence:** Cheaters are not primarily motivated by item drops but by gameplay disruption or rank inflation.

Quantifying the Impact: Cheating and Player Statistics

Beyond anecdotal evidence, empirical data helps us understand the scope of cheating in CS2.

- **Player Growth:** Monthly average players increased from 760k to over 1 million (SteamCharts, Jan 2024 – May 2025).
- **Delayed Enforcement:** 28% of VAC bans occur within 24h of last match; average delay is 20 days.
- **Cheating Embedded:** 97% of affected matches had only one banned player, often undetected in real time.
- **No Case Farming Evidence:** Cheaters are not primarily motivated by item drops but by gameplay disruption or rank inflation.

Quantifying the Impact: Cheating and Player Statistics

Beyond anecdotal evidence, empirical data helps us understand the scope of cheating in CS2.

- **Player Growth:** Monthly average players increased from 760k to over 1 million (SteamCharts, Jan 2024 – May 2025).
- **Delayed Enforcement:** 28% of VAC bans occur within 24h of last match; average delay is 20 days.
- **Cheating Embedded:** 97% of affected matches had only one banned player, often undetected in real time.
- **No Case Farming Evidence:** Cheaters are not primarily motivated by item drops but by gameplay disruption or rank inflation.

Quantifying the Impact: Cheating and Player Statistics

Beyond anecdotal evidence, empirical data helps us understand the scope of cheating in CS2.

- **Player Growth:** Monthly average players increased from 760k to over 1 million (SteamCharts, Jan 2024 – May 2025).
- **Delayed Enforcement:** 28% of VAC bans occur within 24h of last match; average delay is 20 days.
- **Cheating Embedded:** 97% of affected matches had only one banned player, often undetected in real time.
- **No Case Farming Evidence:** Cheaters are not primarily motivated by item drops but by gameplay disruption or rank inflation.

Motivation and Context

The Evolution of Cheating: History, Ecosystem, and Ethics

The Rise of Anti-Cheat: Technologies and Milestones

Counter-Strike 2: A Case Study in Cheating Dynamics

Code Injection and Cheat Delivery Mechanisms

Cheat Implementation: Hooking, Logic, and Features

Reflections, Implications, and Future Work

DLL Injection and Manual Mapping

Cheats are commonly implemented as DLL files injected into the game process.

- **Classic Methods:** LoadLibrary and CreateRemoteThread are frequently used, but easily detected.
- **Manual Mapping:** Avoids standard API calls, skipping LoadLibrary and not registering the module.
- **Stealth Advantage:** Enables loading cheat code directly into memory without leaving traces.
- **Malware Parallels:** These techniques mirror malware behavior, stealthy, evasive, and persistent.

DLL Injection and Manual Mapping

Cheats are commonly implemented as DLL files injected into the game process.

- **Classic Methods:** LoadLibrary and CreateRemoteThread are frequently used, but easily detected.
- **Manual Mapping:** Avoids standard API calls, skipping LoadLibrary and not registering the module.
- **Stealth Advantage:** Enables loading cheat code directly into memory without leaving traces.
- **Malware Parallels:** These techniques mirror malware behavior, stealthy, evasive, and persistent.

DLL Injection and Manual Mapping

Cheats are commonly implemented as DLL files injected into the game process.

- **Classic Methods:** LoadLibrary and CreateRemoteThread are frequently used, but easily detected.
- **Manual Mapping:** Avoids standard API calls, skipping LoadLibrary and not registering the module.
- **Stealth Advantage:** Enables loading cheat code directly into memory without leaving traces.
- **Malware Parallels:** These techniques mirror malware behavior, stealthy, evasive, and persistent.

DLL Injection and Manual Mapping

Cheats are commonly implemented as DLL files injected into the game process.

- **Classic Methods:** LoadLibrary and CreateRemoteThread are frequently used, but easily detected.
- **Manual Mapping:** Avoids standard API calls, skipping LoadLibrary and not registering the module.
- **Stealth Advantage:** Enables loading cheat code directly into memory without leaving traces.
- **Malware Parallels:** These techniques mirror malware behavior, stealthy, evasive, and persistent.

Real-World Implementation: Cheat Loaders

Modern cheat providers rely on custom loaders rather than directly sharing raw DLLs.

- **Encrypted Payloads:** DLLs are downloaded and decrypted at runtime.
- **Anti-Detection Techniques:** Loaders bypass anti-cheat hooks and spoof return addresses.
- **Examples:** Popular cheats like *NeverLose* and *Skeet* deploy secure loaders to ensure stealth.
- **Operational Flow:**

Launch loader → Restart Steam → Inject payload → Execute entry point silently

Real-World Implementation: Cheat Loaders

Modern cheat providers rely on custom loaders rather than directly sharing raw DLLs.

- **Encrypted Payloads:** DLLs are downloaded and decrypted at runtime.
- **Anti-Detection Techniques:** Loaders bypass anti-cheat hooks and spoof return addresses.
- **Examples:** Popular cheats like *NeverLose* and *Skeet* deploy secure loaders to ensure stealth.
- **Operational Flow:**

Launch loader → Restart Steam → Inject payload → Execute entry point silently

Real-World Implementation: Cheat Loaders

Modern cheat providers rely on custom loaders rather than directly sharing raw DLLs.

- **Encrypted Payloads:** DLLs are downloaded and decrypted at runtime.
- **Anti-Detection Techniques:** Loaders bypass anti-cheat hooks and spoof return addresses.
- **Examples:** Popular cheats like *NeverLose* and *Skeet* deploy secure loaders to ensure stealth.
- **Operational Flow:**

Launch loader → Restart Steam → Inject payload → Execute entry point silently

Real-World Implementation: Cheat Loaders

Modern cheat providers rely on custom loaders rather than directly sharing raw DLLs.

- **Encrypted Payloads:** DLLs are downloaded and decrypted at runtime.
- **Anti-Detection Techniques:** Loaders bypass anti-cheat hooks and spoof return addresses.
- **Examples:** Popular cheats like *NeverLose* and *Skeet* deploy secure loaders to ensure stealth.
- **Operational Flow:**

Launch loader → Restart Steam → Inject payload → Execute entry point silently

VAC Detection and Countermeasures

VAC employs traditional and modern detection mechanisms to identify unauthorized modifications.

- **Detection Vectors:** Includes signature scanning, suspicious thread start addresses, and VMT hook detection.
- **Reversing Insight:** Analyzing `steamservice.dll` reveals that VAC streams detection modules from Valve servers.
- **PE Injection:** These modules are manually mapped into memory using PE Injection.
- **Dynamic Landscape:** The result is a constant arms race between detection systems and cheat developers.

VAC Detection and Countermeasures

VAC employs traditional and modern detection mechanisms to identify unauthorized modifications.

- **Detection Vectors:** Includes signature scanning, suspicious thread start addresses, and VMT hook detection.
- **Reversing Insight:** Analyzing `steamservice.dll` reveals that VAC streams detection modules from Valve servers.
- **PE Injection:** These modules are manually mapped into memory using PE Injection.
- **Dynamic Landscape:** The result is a constant arms race between detection systems and cheat developers.

VAC Detection and Countermeasures

VAC employs traditional and modern detection mechanisms to identify unauthorized modifications.

- **Detection Vectors:** Includes signature scanning, suspicious thread start addresses, and VMT hook detection.
- **Reversing Insight:** Analyzing `steamservice.dll` reveals that VAC streams detection modules from Valve servers.
- **PE Injection:** These modules are manually mapped into memory using PE Injection.
- **Dynamic Landscape:** The result is a constant arms race between detection systems and cheat developers.

VAC Detection and Countermeasures

VAC employs traditional and modern detection mechanisms to identify unauthorized modifications.

- **Detection Vectors:** Includes signature scanning, suspicious thread start addresses, and VMT hook detection.
- **Reversing Insight:** Analyzing `steamservice.dll` reveals that VAC streams detection modules from Valve servers.
- **PE Injection:** These modules are manually mapped into memory using PE Injection.
- **Dynamic Landscape:** The result is a constant arms race between detection systems and cheat developers.

Outline

Motivation and Context

The Evolution of Cheating: History, Ecosystem, and Ethics

The Rise of Anti-Cheat: Technologies and Milestones

Counter-Strike 2: A Case Study in Cheating Dynamics

Code Injection and Cheat Delivery Mechanisms

Cheat Implementation: Hooking, Logic, and Features

Reflections, Implications, and Future Work

Hooking the Engine: Input, Rendering, and Events

Hooking allows interception of internal engine calls to alter behavior at runtime.

- **Initial Step:** Reverse engineer the game binary to locate key virtual functions related to input, rendering, or entity updates.
- **Hooking Layer:** Inject trampoline hooks to redirect control flow to our cheat-defined functions.
- **MinHook Library:** We employed MinHook for ease of use and reliability.
- **Limitations:** While MinHook is sufficient for prototyping, more stealth-oriented alternatives exist (e.g., Shadow Hooking, Page Guard, or VEH-based hooks).

Hooking the Engine: Input, Rendering, and Events

Hooking allows interception of internal engine calls to alter behavior at runtime.

- **Initial Step:** Reverse engineer the game binary to locate key virtual functions related to input, rendering, or entity updates.
- **Hooking Layer:** Inject trampoline hooks to redirect control flow to our cheat-defined functions.
- **MinHook Library:** We employed `MinHook` for ease of use and reliability.
- **Limitations:** While `MinHook` is sufficient for prototyping, more stealth-oriented alternatives exist (e.g., Shadow Hooking, Page Guard, or VEH-based hooks).

Hooking the Engine: Input, Rendering, and Events

Hooking allows interception of internal engine calls to alter behavior at runtime.

- **Initial Step:** Reverse engineer the game binary to locate key virtual functions related to input, rendering, or entity updates.
- **Hooking Layer:** Inject trampoline hooks to redirect control flow to our cheat-defined functions.
- **MinHook Library:** We employed MinHook for ease of use and reliability.
- **Limitations:** While MinHook is sufficient for prototyping, more stealth-oriented alternatives exist (e.g., Shadow Hooking, Page Guard, or VEH-based hooks).

Hooking the Engine: Input, Rendering, and Events

Hooking allows interception of internal engine calls to alter behavior at runtime.

- **Initial Step:** Reverse engineer the game binary to locate key virtual functions related to input, rendering, or entity updates.
- **Hooking Layer:** Inject trampoline hooks to redirect control flow to our cheat-defined functions.
- **MinHook Library:** We employed MinHook for ease of use and reliability.
- **Limitations:** While MinHook is sufficient for prototyping, more stealth-oriented alternatives exist (e.g., Shadow Hooking, Page Guard, or VEH-based hooks).

Entity List and Memory Traversal

During the reverse engineering process of CS2, we identified how entities are registered and updated in memory.

- **Entity Management:** The game uses a handle-based paging system to manage all in-game entities.
- **Callback Hooks:** We named the functions `OnAddEntity` and `OnRemoveEntity` during our reverse engineering, as they lack debug symbols. These routines are ideal hook points to track player spawns and despawns.
- **Memory Structures:** Reverse engineering revealed a central list containing pointers to entity classes and their data.

Entity List and Memory Traversal

During the reverse engineering process of CS2, we identified how entities are registered and updated in memory.

- **Entity Management:** The game uses a handle-based paging system to manage all in-game entities.
- **Callback Hooks:** We named the functions `OnAddEntity` and `OnRemoveEntity` during our reverse engineering, as they lack debug symbols. These routines are ideal hook points to track player spawns and despawns.
- **Memory Structures:** Reverse engineering revealed a central list containing pointers to entity classes and their data.

Entity List and Memory Traversal

During the reverse engineering process of CS2, we identified how entities are registered and updated in memory.

- **Entity Management:** The game uses a handle-based paging system to manage all in-game entities.
- **Callback Hooks:** We named the functions `OnAddEntity` and `OnRemoveEntity` during our reverse engineering, as they lack debug symbols. These routines are ideal hook points to track player spawns and despawns.
- **Memory Structures:** Reverse engineering revealed a central list containing pointers to entity classes and their data.

From Theory to Practice: Implementing Cheat Features

Once the game's memory structures and entity management system are fully understood, the implementation of cheat features becomes straightforward.

- **Features Developed:**

- *Chams*: custom materials to highlight player models through geometry.
- *Aimbot*: automatic aim adjustment based on enemy position, using directional vector math and angle normalization (see equations in appendix).
- *Bunny Hop (BHop)*: automated jump timing to maximize movement speed.
- *Visual Enhancements*: overlays and tweaks to highlight relevant data in the scene.
- *Bone ESP*: skeletal rendering of player models to track posture and positioning through obstacles.

From Theory to Practice: Implementing Cheat Features

Once the game's memory structures and entity management system are fully understood, the implementation of cheat features becomes straightforward.

- **Features Developed:**

- *Chams*: custom materials to highlight player models through geometry.
- *Aimbot*: automatic aim adjustment based on enemy position, using directional vector math and angle normalization (see equations in appendix).
- *Bunny Hop (BHop)*: automated jump timing to maximize movement speed.
- *Visual Enhancements*: overlays and tweaks to highlight relevant data in the scene.
- *Bone ESP*: skeletal rendering of player models to track posture and positioning through obstacles.

From Theory to Practice: Implementing Cheat Features

Once the game's memory structures and entity management system are fully understood, the implementation of cheat features becomes straightforward.

- **Features Developed:**

- *Chams*: custom materials to highlight player models through geometry.
- *Aimbot*: automatic aim adjustment based on enemy position, using directional vector math and angle normalization (see equations in appendix).
- *Bunny Hop (BHop)*: automated jump timing to maximize movement speed.
- *Visual Enhancements*: overlays and tweaks to highlight relevant data in the scene.
- *Bone ESP*: skeletal rendering of player models to track posture and positioning through obstacles.

From Theory to Practice: Implementing Cheat Features

Once the game's memory structures and entity management system are fully understood, the implementation of cheat features becomes straightforward.

- **Features Developed:**

- *Chams*: custom materials to highlight player models through geometry.
- *Aimbot*: automatic aim adjustment based on enemy position, using directional vector math and angle normalization (see equations in appendix).
- *Bunny Hop (BHop)*: automated jump timing to maximize movement speed.
- *Visual Enhancements*: overlays and tweaks to highlight relevant data in the scene.
- *Bone ESP*: skeletal rendering of player models to track posture and positioning through obstacles.

From Theory to Practice: Implementing Cheat Features

Once the game's memory structures and entity management system are fully understood, the implementation of cheat features becomes straightforward.

- **Features Developed:**

- *Chams*: custom materials to highlight player models through geometry.
- *Aimbot*: automatic aim adjustment based on enemy position, using directional vector math and angle normalization (see equations in appendix).
- *Bunny Hop (BHop)*: automated jump timing to maximize movement speed.
- *Visual Enhancements*: overlays and tweaks to highlight relevant data in the scene.
- *Bone ESP*: skeletal rendering of player models to track posture and positioning through obstacles.

Chams: Internal Material Injection

To implement Chams, we reverse engineered one of CS2's internal material files to understand its structure.

- **Material Discovery:** We decrypted and analyzed a material from Valve's asset system (suspected format: .vk3).
- **Material Creation:** This allowed us to mimic and reconstruct materials programmatically.
- **Function Hooks:** We hooked two internal engine functions: one responsible for material creation, another for rendering passes.
- **Selective Injection:** By filtering the render queue, we applied our custom material only to specific models (e.g., player entities, weapons).
- **Outcome:** Player models become visually enhanced through walls using internal rendering only.

Chams: Internal Material Injection

To implement Chams, we reverse engineered one of CS2's internal material files to understand its structure.

- **Material Discovery:** We decrypted and analyzed a material from Valve's asset system (suspected format: .vk3).
- **Material Creation:** This allowed us to mimic and reconstruct materials programmatically.
- **Function Hooks:** We hooked two internal engine functions: one responsible for material creation, another for rendering passes.
- **Selective Injection:** By filtering the render queue, we applied our custom material only to specific models (e.g., player entities, weapons).
- **Outcome:** Player models become visually enhanced through walls using internal rendering only.

Chams: Internal Material Injection

To implement Chams, we reverse engineered one of CS2's internal material files to understand its structure.

- **Material Discovery:** We decrypted and analyzed a material from Valve's asset system (suspected format: .vk3).
- **Material Creation:** This allowed us to mimic and reconstruct materials programmatically.
- **Function Hooks:** We hooked two internal engine functions: one responsible for material creation, another for rendering passes.
- **Selective Injection:** By filtering the render queue, we applied our custom material only to specific models (e.g., player entities, weapons).
- **Outcome:** Player models become visually enhanced through walls using internal rendering only.

Chams: Internal Material Injection

To implement Chams, we reverse engineered one of CS2's internal material files to understand its structure.

- **Material Discovery:** We decrypted and analyzed a material from Valve's asset system (suspected format: .vk3).
- **Material Creation:** This allowed us to mimic and reconstruct materials programmatically.
- **Function Hooks:** We hooked two internal engine functions: one responsible for material creation, another for rendering passes.
- **Selective Injection:** By filtering the render queue, we applied our custom material only to specific models (e.g., player entities, weapons).
- **Outcome:** Player models become visually enhanced through walls using internal rendering only.

Chams: Internal Material Injection

To implement Chams, we reverse engineered one of CS2's internal material files to understand its structure.

- **Material Discovery:** We decrypted and analyzed a material from Valve's asset system (suspected format: .vk3).
- **Material Creation:** This allowed us to mimic and reconstruct materials programmatically.
- **Function Hooks:** We hooked two internal engine functions: one responsible for material creation, another for rendering passes.
- **Selective Injection:** By filtering the render queue, we applied our custom material only to specific models (e.g., player entities, weapons).
- **Outcome:** Player models become visually enhanced through walls using internal rendering only.

Visual Examples: Chams Materials



Visual Examples: Chams Materials



Visual Examples: Chams Materials



Visual Examples: Chams Materials



Aimbot: Mathematical Overview

The aimbot determines the angle between the player and the target using 3D vector math. **Angle Calculation:**

$$\text{delta} = \text{targetPos} - \text{playerPos}$$

$$\text{yaw} = \text{atan2}(\text{delta}_y, \text{delta}_x) \times \frac{180}{\pi}$$

$$\text{pitch} = -\text{atan2}(\text{delta}_z, \sqrt{\text{delta}_x^2 + \text{delta}_y^2}) \times \frac{180}{\pi}$$

Explanation:

- Converts a 3D vector into pitch and yaw angles required for aiming.
- Ensures consistent aiming regardless of relative position in space.
- Used each frame to update view angles if target is valid and visible.

Aimbot: Target Selection and Smoothing

For practical gameplay, raw angle calculation is insufficient. We enhance realism and precision using additional heuristics.

- **Interpolation Factor:** We introduce a smoothing coefficient that linearly interpolates between the current view angle and the target angle, reducing abrupt snapping.

- **Formula:**

$$\text{newAngle} = (1 - \alpha) \cdot \text{currentAngle} + \alpha \cdot \text{targetAngle}$$

where $\alpha \in [0, 1]$ controls the smoothing intensity.

- **FOV Filter:** We filter potential targets by their distance to the center of the screen (Field of View), prioritizing those closer to the crosshair.

Aimbot: Target Selection and Smoothing

For practical gameplay, raw angle calculation is insufficient. We enhance realism and precision using additional heuristics.

- **Interpolation Factor:** We introduce a smoothing coefficient that linearly interpolates between the current view angle and the target angle, reducing abrupt snapping.

- **Formula:**

$$\text{newAngle} = (1 - \alpha) \cdot \text{currentAngle} + \alpha \cdot \text{targetAngle}$$

where $\alpha \in [0, 1]$ controls the smoothing intensity.

- **FOV Filter:** We filter potential targets by their distance to the center of the screen (Field of View), prioritizing those closer to the crosshair.

Aimbot: Target Selection and Smoothing

For practical gameplay, raw angle calculation is insufficient. We enhance realism and precision using additional heuristics.

- **Interpolation Factor:** We introduce a smoothing coefficient that linearly interpolates between the current view angle and the target angle, reducing abrupt snapping.

- **Formula:**

$$\text{newAngle} = (1 - \alpha) \cdot \text{currentAngle} + \alpha \cdot \text{targetAngle}$$

where $\alpha \in [0, 1]$ controls the smoothing intensity.

- **FOV Filter:** We filter potential targets by their distance to the center of the screen (Field of View), prioritizing those closer to the crosshair.

Bone ESP: Skeleton Rendering via Bone Structures

During our reverse engineering efforts, we located the in-memory structure used by CS2 to store bone positions for each player model.

- **Bone Matrix Discovery:** The game maintains a matrix of bone positions indexed per entity, storing 3D coordinates for each joint.
- **Rendering Strategy:** Once the matrix was located, we extracted the relevant joint coordinates and projected them into screen space.
- **Skeleton Drawing:** We connected selected bones (e.g., head, spine, limbs) with lines to visually represent the player's skeleton.

Bone ESP: Skeleton Rendering via Bone Structures

During our reverse engineering efforts, we located the in-memory structure used by CS2 to store bone positions for each player model.

- **Bone Matrix Discovery:** The game maintains a matrix of bone positions indexed per entity, storing 3D coordinates for each joint.
- **Rendering Strategy:** Once the matrix was located, we extracted the relevant joint coordinates and projected them into screen space.
- **Skeleton Drawing:** We connected selected bones (e.g., head, spine, limbs) with lines to visually represent the player's skeleton.

Bone ESP: Skeleton Rendering via Bone Structures

During our reverse engineering efforts, we located the in-memory structure used by CS2 to store bone positions for each player model.

- **Bone Matrix Discovery:** The game maintains a matrix of bone positions indexed per entity, storing 3D coordinates for each joint.
- **Rendering Strategy:** Once the matrix was located, we extracted the relevant joint coordinates and projected them into screen space.
- **Skeleton Drawing:** We connected selected bones (e.g., head, spine, limbs) with lines to visually represent the player's skeleton.

Visual Examples: Bone Matrix



Visual Examples: Skeleton



Visual Examples: Skeleton & Chams in a Real Match



Visual Examples: Skeleton in a Real Match



Outline

Motivation and Context

The Evolution of Cheating: History, Ecosystem, and Ethics

The Rise of Anti-Cheat: Technologies and Milestones

Counter-Strike 2: A Case Study in Cheating Dynamics

Code Injection and Cheat Delivery Mechanisms

Cheat Implementation: Hooking, Logic, and Features

Reflections, Implications, and Future Work

Final Reflections

- Cheating in online games reflects a complex interplay between technical ingenuity and social dynamics.
- The continual arms race between cheat developers and anti-cheat systems mirrors broader cybersecurity challenges.
- Gaining offensive insight through cheat implementation reveals critical blind spots in detection strategies.
- Ethical, economic, and technical dimensions must be addressed together to preserve competitive integrity.

Final Reflections

- Cheating in online games reflects a complex interplay between technical ingenuity and social dynamics.
- The continual arms race between cheat developers and anti-cheat systems mirrors broader cybersecurity challenges.
- Gaining offensive insight through cheat implementation reveals critical blind spots in detection strategies.
- Ethical, economic, and technical dimensions must be addressed together to preserve competitive integrity.

Final Reflections

- Cheating in online games reflects a complex interplay between technical ingenuity and social dynamics.
- The continual arms race between cheat developers and anti-cheat systems mirrors broader cybersecurity challenges.
- Gaining offensive insight through cheat implementation reveals critical blind spots in detection strategies.
- Ethical, economic, and technical dimensions must be addressed together to preserve competitive integrity.

Final Reflections

- Cheating in online games reflects a complex interplay between technical ingenuity and social dynamics.
- The continual arms race between cheat developers and anti-cheat systems mirrors broader cybersecurity challenges.
- Gaining offensive insight through cheat implementation reveals critical blind spots in detection strategies.
- Ethical, economic, and technical dimensions must be addressed together to preserve competitive integrity.

Future Work and Research Directions

- Extend the cheat framework to support multiple Source 2 titles, enabling comparative analysis.
- Integrate automated schema extraction and dynamic offset resolution for update resilience.
- Design and test new stealth hooking methods, including hardware breakpoints and kernel-level indirection.
- Explore cheat detection benchmarking using controlled environments and real-time telemetry.

Future Work and Research Directions

- Extend the cheat framework to support multiple Source 2 titles, enabling comparative analysis.
- Integrate automated schema extraction and dynamic offset resolution for update resilience.
- Design and test new stealth hooking methods, including hardware breakpoints and kernel-level indirection.
- Explore cheat detection benchmarking using controlled environments and real-time telemetry.

Future Work and Research Directions

- Extend the cheat framework to support multiple Source 2 titles, enabling comparative analysis.
- Integrate automated schema extraction and dynamic offset resolution for update resilience.
- Design and test new stealth hooking methods, including hardware breakpoints and kernel-level indirection.
- Explore cheat detection benchmarking using controlled environments and real-time telemetry.

Future Work and Research Directions

- Extend the cheat framework to support multiple Source 2 titles, enabling comparative analysis.
- Integrate automated schema extraction and dynamic offset resolution for update resilience.
- Design and test new stealth hooking methods, including hardware breakpoints and kernel-level indirection.
- Explore cheat detection benchmarking using controlled environments and real-time telemetry.

Thank you for your attention!

Any questions?